KEMET

AI Egypt Tourism Assistant

GRADUATION PROJECT DOCUMENTATION

An AI-Powered Platform for Egyptian Tourism, Heritage & Trip Planning

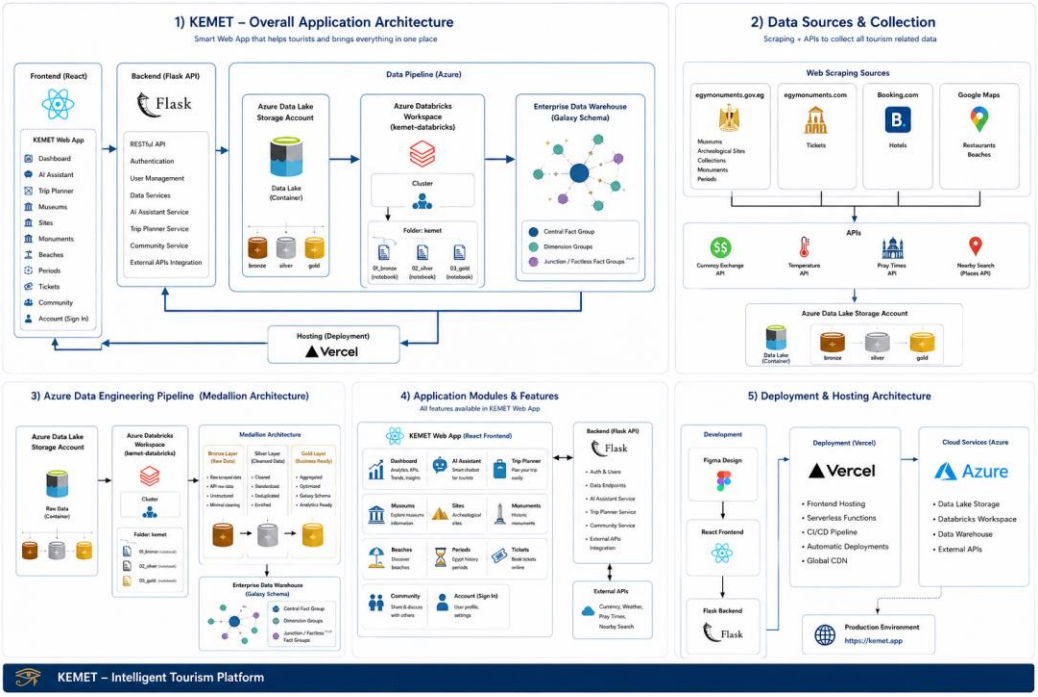
Submitted By

Muhammed Ayman Elsayegh
Elsayed Ashraf Bakry
Ahmed Hassan Mohamed
Yousef Hamdy Yassin

1. Project Planning & Management

1.1 Project Overview

Kemet is an AI-powered web platform that helps travelers explore Egypt's tourism, heritage and hospitality landscape from a single interface. Deployed as a Docker-based Hugging Face Space, it combines a conversational AI assistant, a live tourism dashboard, curated directories of hotels, restaurants, museums, monuments, archaeological sites and historical periods, and a lightweight community feed, all wrapped in a dark, gold-accented interface themed around ancient Egyptian identity. Behind the front end, a dedicated data engineering pipeline (Azure Data Factory, Azure Databricks, and a Flask API layer) collects, cleans and models the tourism datasets that power both the directories and the AI assistant — see section 4.8.



Overview of the Kemet application — chatbot, dashboard and directory pages

1.2 Objectives & Scope

Objectives:

- Provide a single AI assistant that can answer general Egypt-travel questions, search the live web, and answer questions grounded in curated tourism datasets (RAG).
- Support both Arabic and English input/output with automatic right-to-left / left-to-right rendering.
- Surface live, practical information: weather across major destinations, currency exchange rates, emergency numbers and useful travel apps.
- Offer browsable, filterable directories of hotels, restaurants, museums, monuments, archaeological sites and historical periods.
- Allow authenticated users to create posts, keep a personal profile, and browse a community feed.

Scope:

- Conversational AI assistant (Chat / Web Search / Data modes) powered by Google Gemini.

- Retrieval-Augmented Generation (RAG) over CSV/Excel datasets stored in Azure Blob Storage.
- A cloud-native data engineering pipeline — Selenium/API-based collection, Azure Data Factory orchestration, Azure Databricks processing through a Medallion (Bronze/Silver/Gold) architecture, and a Galaxy Schema analytics model — exposed to the platform via Flask REST APIs.
- Live tourism analytics dashboard with weather, arrivals and nationality charts.
- Directory browsing modules for points of interest across Egypt.
- User accounts, posts and community features backed by Azure Cosmos DB.

1.3 Project Plan & Timeline

Suggested phase breakdown for the graduation timeline:

Phase	Focus	Key Deliverables
Phase 1 — Planning & Setup	Requirements, UX direction, repo scaffolding	Project proposal, application skeleton, Kemet theme
Phase 2 — Data Collection	Web scraping (Selenium) & public APIs for hotel / restaurant / museum / monument data	Raw datasets landed in the data lake (Bronze layer)
Phase 3 — Data Engineering	Azure Data Factory orchestration, Azure Databricks cleaning & modeling (Medallion architecture), Galaxy Schema design	Curated Gold-layer datasets, Flask REST API
Phase 4 — Core Directories	Hotels, Restaurants, Museums, Monuments, Sites, Periods	Filterable directory pages
Phase 5 — AI Assistant	Gemini integration, RAG pipeline, web search fallback	Chat / Web Search / Data chatbot modes
Phase 6 — Dashboard	Live weather, tourism stats, currency, emergency info	Interactive dashboard page
Phase 7 — Accounts & Community	Cosmos DB auth, posts, profile	Account, Your Posts, Community pages
Phase 8 — Containerization & Deployment	Dockerfile, Hugging Face Space config	Live Space on Hugging Face
Phase 9 — Testing & Documentation	QA pass, this documentation	Test report, final documentation

1.4 Task Assignment & Roles

Team Member	Role	Key Responsibilities
Muhammed Ayman Elsayegh	Team Leader / AI Engineer	Gemini integration, RAG pipeline, chatbot architecture
Elsayed Ashraf Bakry	Data Engineer	Web-scraping/API data collection, Azure Data Factory & Databricks pipeline, Medallion

Team Member	Role	Key Responsibilities
		architecture, Galaxy Schema design, Azure Blob Storage & Cosmos DB setup
Ahmed Hassan Mohamed	Frontend / UI Developer	Application theming, dashboard, directory pages
Yousef Hamdy Yassin	Backend Developer	Flask REST API middleware, Cosmos DB-based authentication, account & community features

1.5 Key Performance Indicators (KPIs)

KPI	Target	Measurement Method
Chatbot response time	< 5 seconds	Time from question submit to rendered reply
RAG retrieval relevance	Top-4 chunks on-topic	Manual spot-check of retrieved chunks vs. query
Bilingual accuracy (AR/EN)	Correct language + direction	Manual review of RTL/LTR rendering
Dashboard data freshness	≤ 5 minutes	Weather auto-refresh interval
Data pipeline freshness	Daily Gold-layer refresh	Azure Data Factory pipeline run monitoring
System uptime (HF Space)	≥ 99%	Hugging Face Space status monitoring

2. Literature Review

2.1 Background Research

Tourism Information Challenges Addressed:

- Travel information about Egypt is scattered across many sources (blogs, forums, government sites, booking platforms), making trip planning slow and inconsistent.
- Language barriers: most detailed heritage content is written in English, while a large share of visitors and local users need Arabic support.
- Practical, time-sensitive information (weather, currency rates, emergency numbers) is rarely combined with cultural/heritage content in one place.
- Generic chatbots are not grounded in curated, up-to-date local datasets and can hallucinate details about specific hotels, sites or museums.

Technology Review:

- Large Language Models (Google Gemini 2.5 Flash / Flash-Lite) provide strong general-purpose reasoning and can be grounded with tools such as Google Search or custom retrieval.
- Retrieval-Augmented Generation (RAG) with sentence embeddings lets an assistant answer from proprietary datasets instead of relying purely on model memory, reducing hallucination.
- Multilingual sentence-transformer models (paraphrase-multilingual-MiniLM-L12-v2) allow a single embedding space to serve both Arabic and English queries accurately.
- Cloud object storage (Azure Blob Storage) is a cost-effective way to host and update structured tourism datasets without redeploying the application.
- A Medallion architecture (Bronze / Silver / Gold layers) on Azure Databricks, orchestrated by Azure Data Factory, is a widely used pattern for turning raw scraped/API data into clean, analytics-ready datasets.
- A Galaxy Schema (multiple fact tables sharing conformed dimensions) supports fast analytical queries and keeps reporting, dashboards and AI retrieval consistent with one another.

2.2 Technology Justification

Technology	Justification
Google Gemini (google-genai)	State-of-the-art multilingual LLM with native Google Search grounding for the general Chat mode.
sentence-transformers	Local, free multilingual embeddings for the RAG retrieval pipeline (no external embedding API cost).
Selenium	Automates web scraping of hotel, restaurant, museum and monument listings from trusted public sources into the raw (Bronze) data layer.
Azure Data Factory	Orchestrates and schedules ingestion of scraped and API-sourced data into the pipeline.
Azure Databricks	Cleans, transforms and organizes raw data through the Bronze → Silver → Gold Medallion architecture and builds the Galaxy Schema model.

Technology	Justification
Flask (REST APIs)	Lightweight middleware exposing the curated Gold-layer data and AI services from the data pipeline to the Kemet platform and any other consuming module.
Azure Blob Storage	Central, updatable data lake for the CSV/Excel datasets that power both directories and RAG retrieval.
Azure Cosmos DB	Flexible, low-latency document database used both for structured application data and for custom user authentication, profiles, posts and the community feed.
ddgs (DuckDuckGo Search)	Free, keyless web search backend for the assistant's Web Search mode.
Plotly / Chart.js	Interactive charts for the tourism dashboard (arrivals, nationalities, weather).
Docker	Reproducible container image required by the Hugging Face Spaces deployment SDK.

3. Requirements Gathering

3.1 Stakeholder Analysis

Stakeholder	Needs	Priority
Tourists / Travelers	Trustworthy trip info, AI assistant, weather & currency	High
Local & Foreign Visitors (Arabic/English)	Bilingual, culturally aware answers	High
Hotel / Restaurant Owners	Visibility in the directory listings	Medium
Community Members	Ability to post, follow trends, engage with other travelers	Medium
System Administrators	Reliable deployment, manageable datasets, monitored API usage	Medium

3.2 User Stories & Use Cases

User Story 1: AI Trip Assistant

- As a tourist, I want to ask an AI assistant questions about Egypt in Arabic or English, so that I can plan my trip without searching multiple websites.

User Story 2: Grounded Recommendations

- As a traveler, I want the assistant's hotel/restaurant/museum answers to be grounded in a real dataset, so that I get accurate, non-hallucinated details.

User Story 3: Live Travel Conditions

- As a traveler, I want to see live weather across major Egyptian cities and current currency exchange rates, so that I can plan my days and budget.

User Story 4: Directory Browsing

- As a user, I want to browse hotels, restaurants, museums, monuments, sites and historical periods with filters, so that I can discover options relevant to my trip.

User Story 5: Community Engagement

- As a registered user, I want to create an account, write posts, and view a community feed, so that I can share and discover travel experiences.

4. System Analysis & Design

4.1 Problem Statement & Objectives

Problem Statement: Planning a trip to Egypt currently requires consulting many disconnected sources — travel blogs, booking sites, museum listings, weather apps and currency converters — most of which are not tailored to Arabic-speaking users and none of which combine an AI assistant with curated local data. This fragmentation increases planning time and the risk of outdated or inaccurate information.

Project Objectives:

- Provide a unified, bilingual AI assistant with three complementary modes: general chat, live web search, and dataset-grounded RAG answers.
- Build a cloud-native data engineering pipeline that turns raw scraped/API tourism data into clean, analytics-ready Gold-layer data, modeled as a Galaxy Schema and served through Flask REST APIs.
- Aggregate practical, time-sensitive data (weather, currency, emergency numbers) into one live dashboard.
- Offer structured, filterable directories for the main Egyptian tourism categories.
- Support user accounts and a lightweight community feed for shared travel knowledge.
- Package the whole application as a portable Docker container for simple cloud deployment.

4.2 System Architecture

Architecture Style: A cloud-native data engineering pipeline feeds a modular web application through a Flask API layer, with an external AI layer (Gemini) and dual storage (Azure Blob Storage + Azure Cosmos DB).

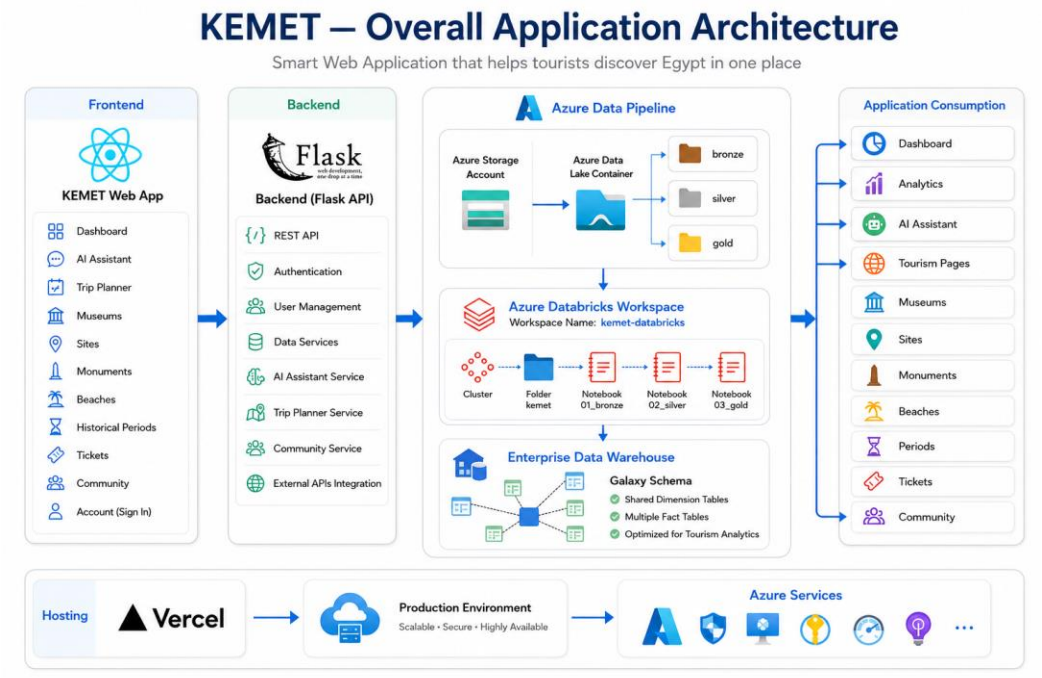


Figure 4.2 — End-to-end system architecture, from raw data sources to the deployed application

Note: the earlier version of this document described the application as reading directly from Azure Blob Storage / Cosmos DB, without the upstream data engineering pipeline. This section has been corrected to match the pipeline

presented in the project's slide deck: raw data is first collected and processed through Azure Data Factory, Azure Databricks and a Galaxy Schema model, then served to the platform via a Flask REST API. See section 4.8 for details.

4.3 Use Case Overview

Primary Actors: Guest User, Registered User, System Administrator.

Key Use Cases:

- Chat with the AI assistant (Chat / Web Search / Data modes)
- View live weather, currency exchange and tourism statistics
- Browse and filter hotels, restaurants, museums, monuments, sites and periods
- Search Muslim-traveler information (e.g. prayer-related and halal considerations)
- Create an account, sign in and manage a profile
- Publish a post and view the community feed

4.4 Data Model & Data Sources

Kemet's content is driven by a set of curated CSV/Excel datasets, produced by the Gold layer of the data engineering pipeline (section 4.8) and stored in an Azure Blob Storage container (sourcedatalake). These datasets are used both for the directory pages and as the RAG knowledge base for the chatbot's Data mode:

Dataset	Powers
Egypt_All_Governorates_Hotels.csv	Hotels directory + hotel-related chatbot answers
restaurants_gmaps.csv	Restaurants directory + restaurant-related chatbot answers
museums_en.csv	Museums directory + museum-related chatbot answers
monuments_en.csv	Monuments directory + monument-related chatbot answers
Ancient_Sites_En.csv	Archaeological sites directory + site-related chatbot answers
collections_en.csv	Museum collections / artifacts referenced by the chatbot
periods_en.csv	Historical periods directory + dynasty/period chatbot answers

User-generated data (accounts, posts, community content) and structured application data are both stored in Azure Cosmos DB, which also backs the platform's custom user-authentication flow (replacing the team's earlier Firebase-based prototype).

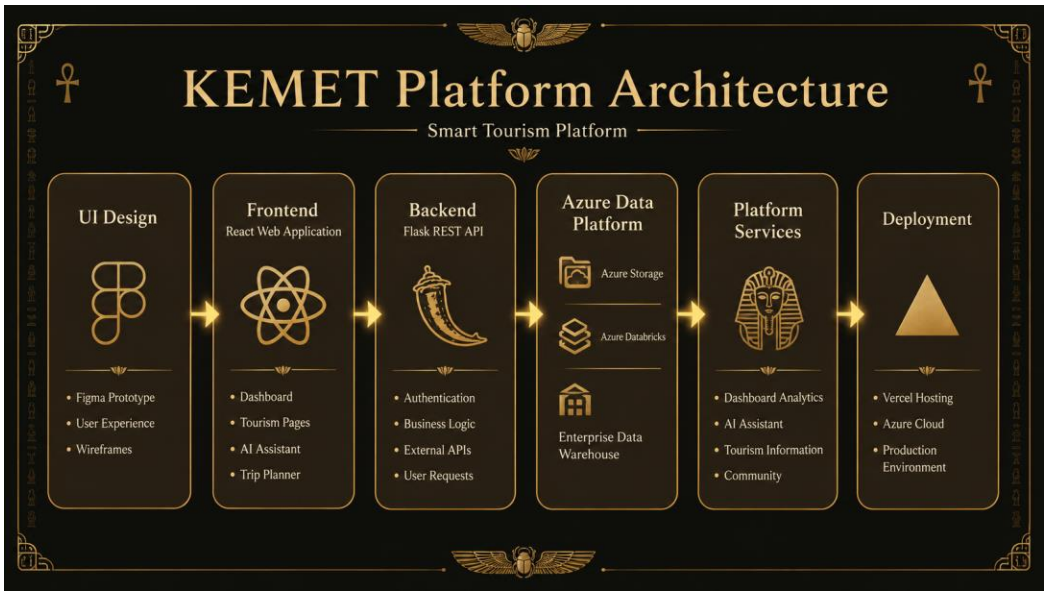


Figure 4.4a — Complete Gold-layer schema covering all curated tourism datasets

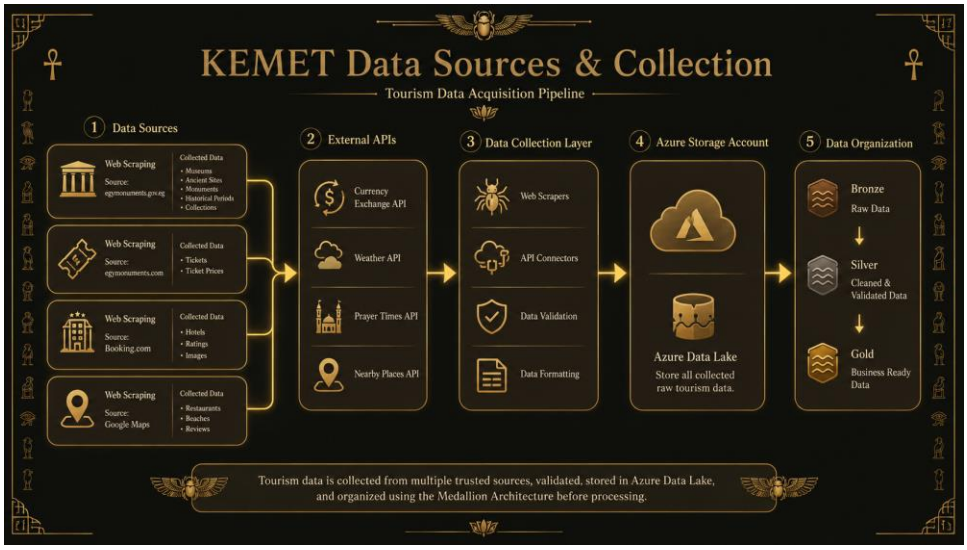


Figure 4.4b — Gold-layer detail for the museum collections dataset

4.5 Chatbot / RAG Data Flow

Data mode question-answering flow:

- User Question (AR/EN)
- Keyword Router — detect_relevant_sources() narrows the dataset search space
- Chunking — CSV rows (12/chunk) or text (800 chars/chunk), cached per file
- Multilingual Embeddings — paraphrase-multilingual-MiniLM-L12-v2, cosine similarity
- Top-K Retrieval — 4 most relevant chunks selected as context
- Gemini 2.5 (Flash / Flash-Lite) — generates grounded answer from retrieved context
- Directional Rendering — auto RTL for Arabic, LTR for English

4.6 Technology Stack

Layer	Technology	Purpose
Data Collection	Selenium, public APIs	Gathers raw hotel/restaurant/museum/monument data
Data Orchestration	Azure Data Factory	Schedules and orchestrates ingestion into the pipeline
Data Processing	Azure Databricks	Medallion architecture (Bronze/Silver/Gold) + Galaxy Schema modeling
API Middleware	Flask	RESTful APIs delivering curated data & AI services to the platform
Frontend / UI	Python-based web application framework	Multi-page web app + themed sidebar navigation
AI — LLM	Google Gemini 2.5 Flash / Flash-Lite (google-genai)	Conversational answers, Google-Search-grounded chat
AI — Embeddings	sentence-transformers (multilingual MiniLM)	RAG retrieval over local datasets
Web Search	ddgs (DuckDuckGo Search)	Free, keyless real-time web search mode
Data Lake	Azure Blob Storage	CSV/Excel datasets for directories & RAG
Database & Auth	Azure Cosmos DB	Structured application data, custom user authentication, posts and community feed
Visualization	Plotly, Chart.js	Dashboard charts (weather, arrivals, nationalities)
External APIs	Open-Meteo	Live weather per city
Containerization	Docker	Deployment image for Hugging Face Spaces
Hosting	Hugging Face Spaces (Docker SDK)	Public hosting

4.7 Deployment Architecture

Kemet is packaged as a Docker image and hosted as a Hugging Face Space configured with the docker SDK and a defined application entry point. Configuration values (Gemini API key, Azure Blob connection string, Cosmos DB endpoint/key, Flask API base URL) are supplied via Space secrets / environment variables (kept out of source control via .gitignore) and read at runtime through a small get_secret() helper.

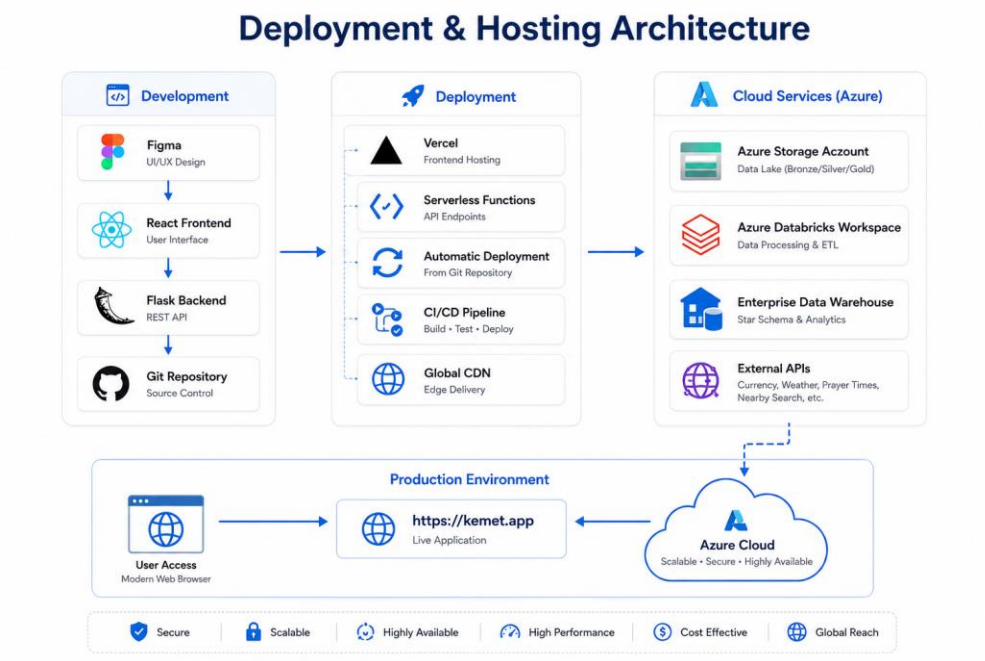


Figure 4.7 — Deployment flow: container build to live Hugging Face Space

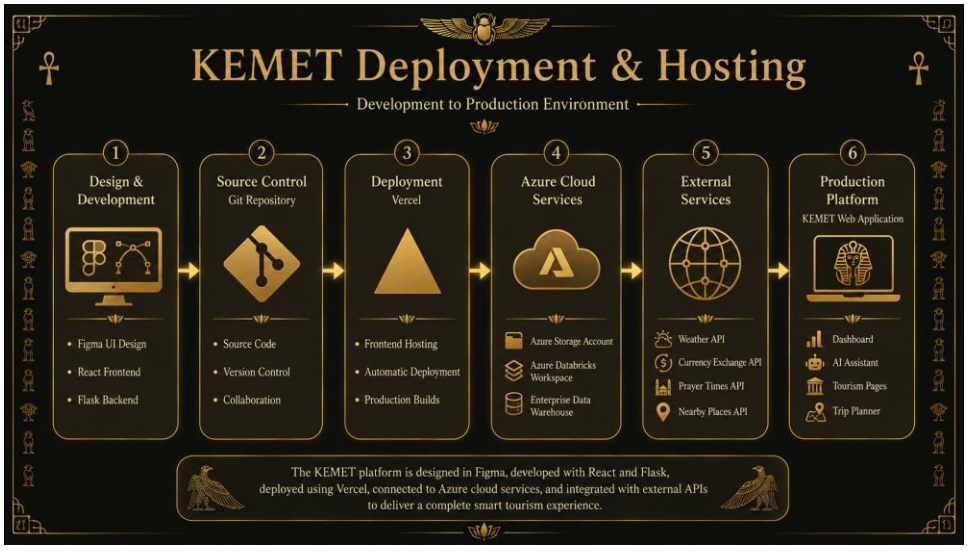


Figure 4.7b — Gold-layer schema detail supporting the deployed application

4.8 Data Engineering Pipeline & Analytics Schema

Kemet is powered by a cloud-native data engineering pipeline that collects tourism data from multiple trusted sources, processes it using Azure Data Factory and Azure Databricks, and organizes it through the Medallion Architecture before serving clean, analytics-ready data to the platform's Flask APIs and AI services.

Data Collection & Ingestion

Raw data is gathered from trusted sources using web scraping (Selenium) and public APIs, creating a reliable and continuously updated knowledge base covering hotels & accommodation, restaurants & local dining, museums, historical landmarks, monuments and archaeological sites.

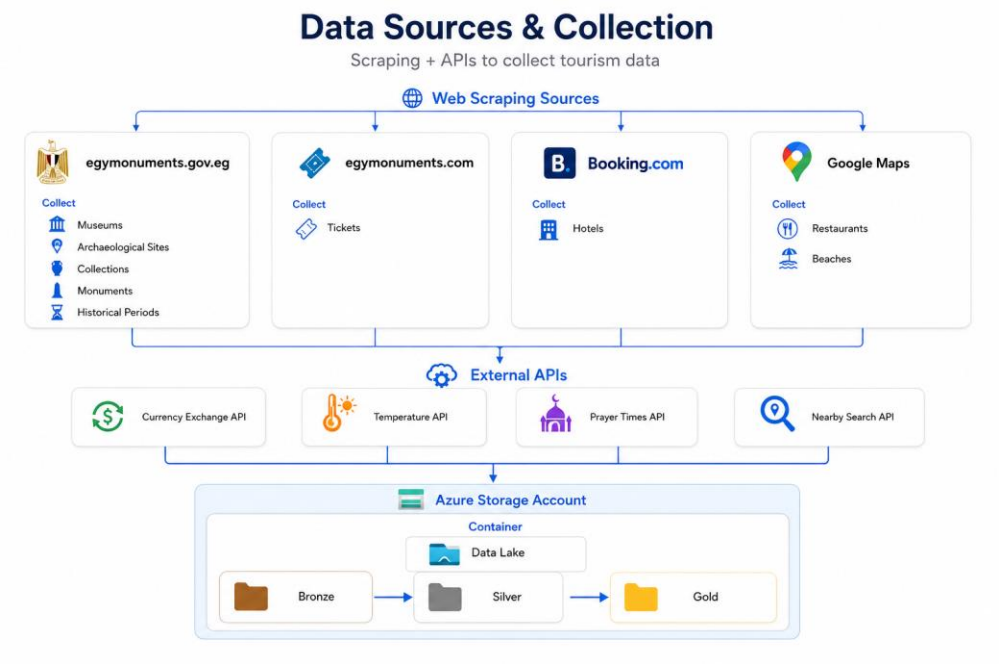


Figure 4.8a — Data collection: web scraping and public APIs feeding the Bronze layer

Medallion Architecture

- Bronze layer — raw, unprocessed data as collected from scraping and APIs.
- Silver layer — cleaned, validated and de-duplicated data.
- Gold layer — curated, analytics-ready data modeled for reporting, dashboards and the AI assistant.

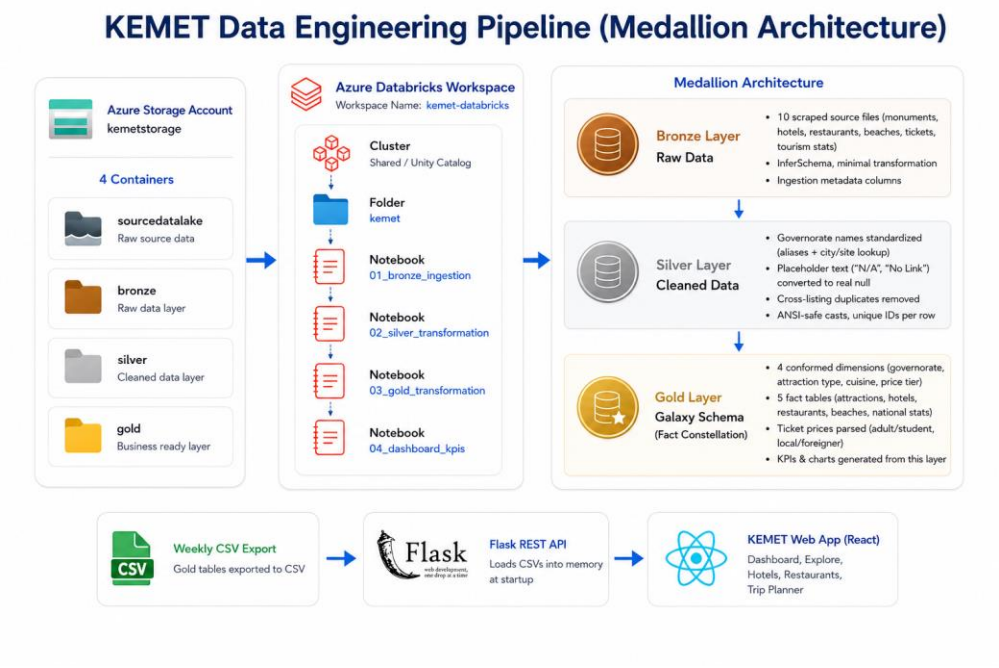


Figure 4.8b — Medallion architecture: Bronze → Silver → Gold

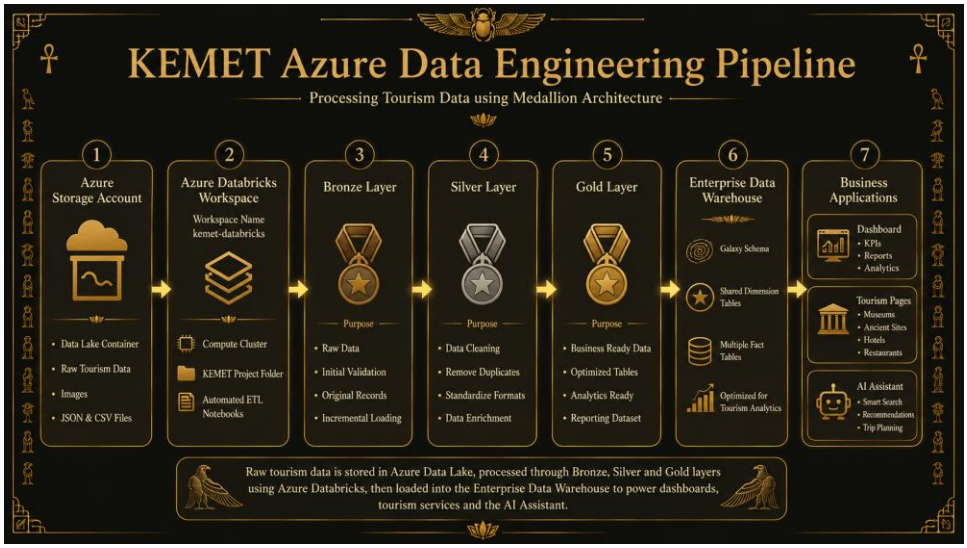


Figure 4.8c — Gold-layer detail within the Medallion pipeline

Galaxy Schema

The Gold layer is modeled as a Galaxy Schema — a data-warehouse design where multiple fact tables share a set of conformed dimension tables (e.g. location, category, date). Key benefits:

- Fast analytical queries
- Centralized tourism data
- Simplified reporting
- Scalable data model
- Supports AI & dashboards

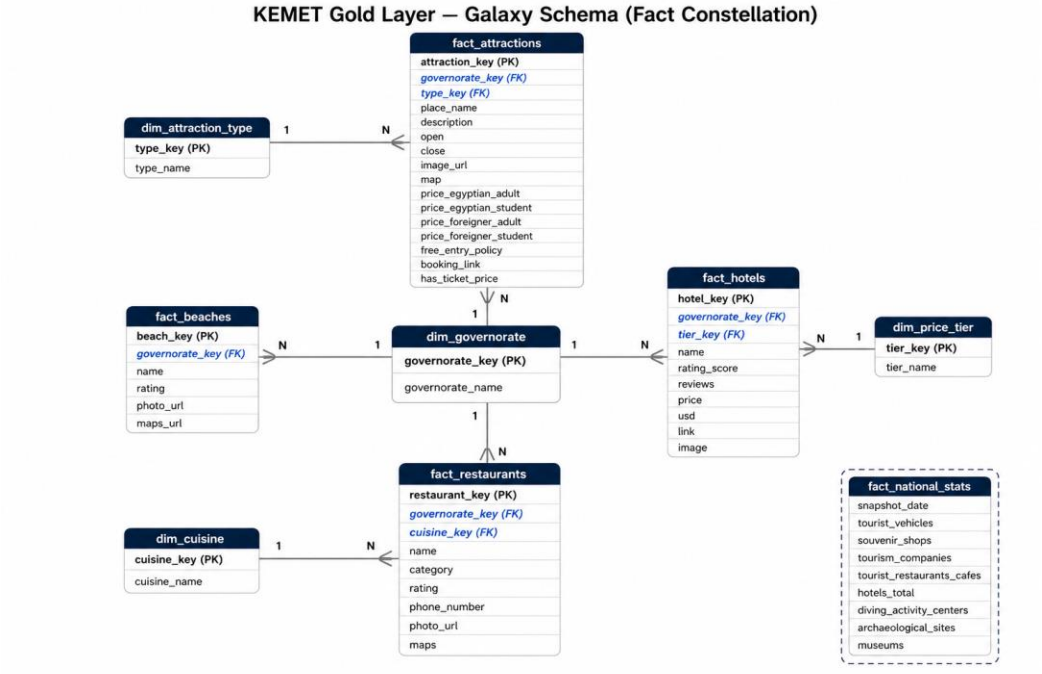


Figure 4.8d — Galaxy Schema data model powering Kemet's analytics layer

Flask API Middleware

Flask acts as the middleware between the Gold layer and the Kemet platform, exposing secure RESTful APIs that deliver processed tourism data and AI services to every application module — including the directory pages and, indirectly, the datasets consumed by the chatbot's RAG pipeline.

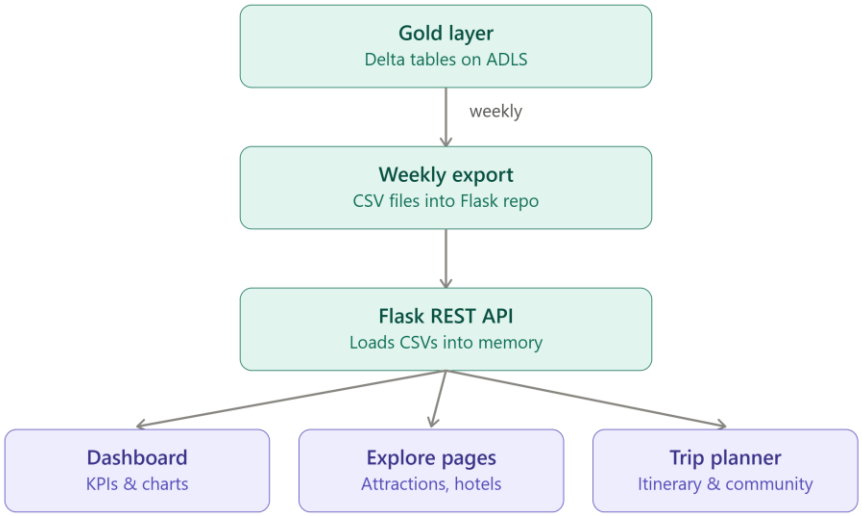


Figure 4.8e — Gold layer to Flask API data flow

5. Implementation Details

This chapter explains how Kemet was implemented across its data pipeline, UI shell, AI assistant, dashboard, directory pages, community features and deployment pipeline.

5.1 Project Structure

Path	Description
config.toml	Application configuration (theme, server settings)
secrets.toml	Local secrets file (Gemini API key, Azure & Cosmos DB credentials, Flask API URL) — git-ignored
.gitignore	Excludes .venv, __pycache__ and secrets.toml from version control
app.py	Application entry point: page config, global theme CSS, sidebar navigation and page routing
utils.py	get_secret() helper for reading environment/secret values
avatar_utils.py	Helper functions for generating and rendering user avatar images
cookie_utils.py	Browser-cookie helpers used to persist the logged-in session across reloads
src/home.py	Community feed ("Community" page)
src/your.py	"Your Posts" — the current user's published content
src/account.py	Sign up / sign in / profile management (Azure Cosmos DB-backed auth)
src/support.py	Help / support and contact page
src/dashboard.py	Live tourism dashboard: stats, charts, weather, currency, emergency info
src/chatbot.py	AI assistant: Chat / Web Search / Data (RAG) modes
src/hotels.py	Hotels directory
src/restaurants.py	Restaurants directory
src/museums.py	Museums directory
src/monuments.py	Monuments directory
src/sites.py	Archaeological sites directory
src/periods.py	Historical periods directory
src/muslim.py	Muslim-traveler information
requirements.txt	Python dependencies

The data engineering side of Kemet — the Azure Data Factory / Databricks pipeline, the Galaxy Schema warehouse, and the Flask REST API middleware described in section 4.8 — lives in a separate backend service outside this application's app.py/src/ front-end structure; the front end consumes its output via Azure Blob Storage and the Flask API.

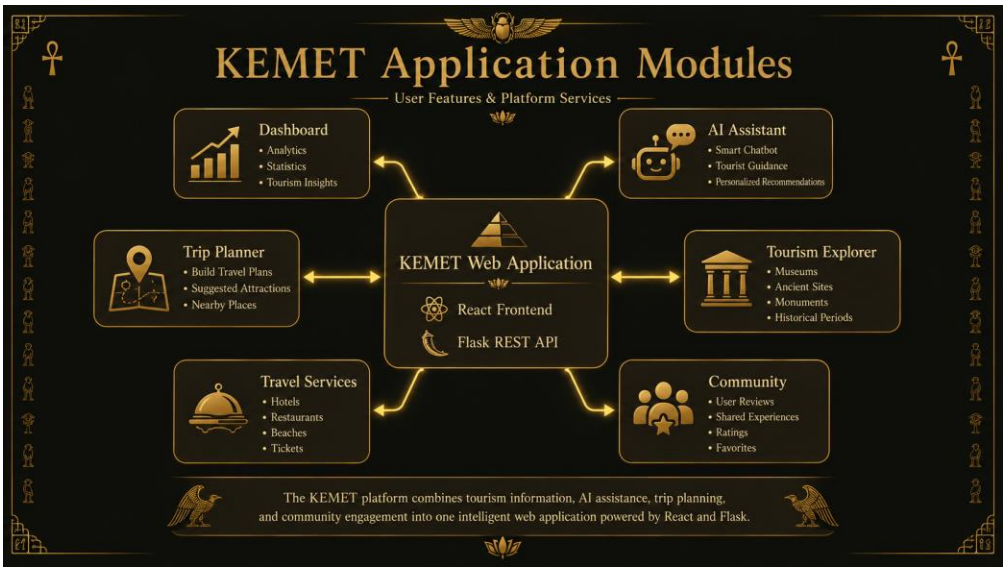


Figure 5.1 — Kemet application modules

5.2 UI Shell & Theming

app.py defines a single global stylesheet, applied across every page, that gives the whole application a consistent identity:

- Dark navy background (#0a0c18) with card surfaces in #161929 and #1e2235, bordered in #2c3248.
- Gold accent (#dfb257 → #c49a3c gradient) used for the active sidebar item, buttons, links and highlighted values — evoking ancient Egyptian gold artifacts.
- Inter font family, imported from Google Fonts, applied globally for a clean, modern look.
- A custom sidebar header combining the Eye of Horus glyph (𓂀) with the "KEMET" wordmark and tagline "AI Egypt Tourism Assistant".
- Navigation implemented with a custom sidebar menu component (Community, Your Posts, Dashboard, Chatbot, Hotels, Restaurants, Muslim, Sites, Museums, Monuments, Periods, Support, Account), with default framework menu/footer/header hidden for a native app feel.
- A fixed bottom-of-sidebar user card showing the signed-in username (or "Guest User") and plan label.

5.3 Chatbot Module

Implemented in src/chatbot.py, the assistant offers three selectable modes:

Mode	How it works
Chat	Calls Gemini with the Google Search tool enabled for grounded, general-purpose answers about Egypt tourism.
Web Search	Uses ddgs to fetch live search results (with backend fallback and retry logic to survive rate limits), then asks Gemini to answer from those results — using no Gemini "grounding" quota.
Data	Runs the RAG pipeline described in section 4.5 against the Azure-hosted datasets and asks Gemini to answer only from the retrieved excerpts.

Key implementation details:

- A keyword router (detect_relevant_sources) maps question terms — in Arabic or English — to the most relevant dataset file(s) before running embedding search, keeping the context small and the answers focused.
- Chunk embeddings are computed once and cached in memory with a 1-hour TTL, so repeated questions are fast and don't re-embed the whole dataset.
- Two Gemini model tiers are offered — "Flash" (default balance of speed/quality) and "Flash Lite" (faster, higher free-tier limits) — selectable per conversation.
- A minimum 4-second gap is enforced between requests, and 429 (rate limit) errors are caught and explained to the user in Arabic, distinguishing per-minute limits (auto-retried once) from daily quota limits (with guidance to switch modes/models).
- detect_text_direction() inspects Arabic vs. Latin character counts to automatically render each chat bubble right-to-left or left-to-right, so numbered lists, bullets and bold text align correctly for the detected language.

5.4 Dashboard Module

Implemented in src/dashboard.py using a mix of embedded HTML/JS (for a Chart.js-driven stats/charts section) and native Plotly chart components:

- Live weather for ten major destinations (Cairo, Alexandria, Giza, Luxor, Aswan, Hurghada, Sharm El Sheikh, Dahab, Saint Catherine, Fayoum) fetched from the free Open-Meteo API, refreshed automatically every 5 minutes.
- Summary cards for hottest / coolest / most humid / driest city, plus horizontal Plotly bar charts ranking all cities by temperature and humidity.
- A Chart.js bar chart of tourist arrivals (2016–2025) and a doughnut chart of top visitor nationalities for 2025.
- Emergency information (tourist police, ambulance, fire, embassy hotline), a currency exchange snapshot (EGP vs. USD/EUR/GBP/SAR/AED), and quick links to useful traveler apps (ride-hailing, food delivery, health, maps).
- A manual "Refresh Weather Now" button plus an automatic rerun once the 5-minute refresh interval elapses.

5.5 Directory Modules

Hotels, Restaurants, Museums, Monuments, Sites and Periods each follow the same pattern: load the relevant Gold-layer dataset (via the Flask API / Azure Blob Storage), present it as a searchable/filterable view styled with the shared Kemet theme, and surface the same underlying data that powers the chatbot's Data mode — keeping directory browsing and AI answers consistent. A dedicated Muslim module surfaces information relevant to Muslim travelers.

5.6 Community & Account Modules

- src/account.py — sign-up / sign-in and profile management backed by a custom Azure Cosmos DB user store (session persistence handled via cookie_utils.py).
- src/home.py — the "Community" landing feed of shared posts.
- src/your.py — "Your Posts", the signed-in user's own published content.
- src/support.py — help / support and contact page for users.
- avatar_utils.py — helper functions for generating and rendering user avatar images.
- cookie_utils.py — browser-cookie helpers that persist the logged-in session across page reloads.

5.7 Data & Storage Layer

Store	Role
Azure Data Factory + Databricks (Bronze/Silver/Gold)	Upstream pipeline that collects and curates raw tourism data into analytics-ready Gold-layer datasets
Azure Blob Storage (sourcedatalake)	Source of truth for the curated CSV/Excel tourism datasets used by directories and RAG
Azure Cosmos DB	User authentication, profiles, posts, community content and other structured application data

5.8 Testing & Execution

Local development:

- python app.py — runs the full multi-page app locally

Container build & run (matches the Hugging Face Spaces Docker SDK configuration):

- docker build -t kemet .
- docker run -p 8501:8501 --env-file .env kemet

Required secrets (environment variables / Space secrets): GEMINI_API_KEY, AZURE_DATALAKE_CONNECTION_STRING, the Azure Cosmos DB endpoint and key, and the Flask API base URL used for authentication and app data.

6. Testing & Quality Assurance

6.1 Key Test Scenarios

Scenario 1 — Chatbot Modes

- Verify Chat mode returns Google-Search-grounded answers for general Egypt-travel questions.
- Verify Web Search mode returns fresh results even under DuckDuckGo rate limiting (backend fallback/retry).
- Verify Data mode retrieves chunks from the correct dataset for Arabic and English keyword queries.

Scenario 2 — Bilingual Rendering

- Confirm Arabic answers render right-to-left with correctly ordered lists and bold text.
- Confirm English answers render left-to-right.

Scenario 3 — Rate Limiting & Error Handling

- Confirm the 4-second request throttle triggers the Arabic wait message.
- Confirm per-minute 429 errors are retried once automatically.
- Confirm daily-quota 429 errors surface the guidance message instead of retrying indefinitely.

Scenario 4 — Dashboard

- Confirm weather cards and charts refresh automatically every 5 minutes and via the manual refresh button.
- Confirm hottest/coolest/most-humid/driest calculations match the underlying data.

Scenario 5 — Directories & Accounts

- Confirm each directory page loads its dataset and filters correctly.
- Confirm sign-up, sign-in and post creation work end-to-end through Azure Cosmos DB.

Scenario 6 — Data Pipeline

- Confirm Azure Data Factory pipeline runs complete successfully and land data in the Bronze layer.
- Confirm Databricks transformations correctly populate the Silver and Gold layers with no data loss.
- Confirm the Flask API returns the expected Gold-layer records for each directory endpoint.

7. Results & Impact

7.1 Key Achievements

- A single AI assistant covering three complementary answer strategies (general chat, live web search, dataset-grounded RAG).
- Native bilingual support (Arabic/English) with automatic text-direction detection.
- A cloud-native data engineering pipeline (Azure Data Factory + Databricks, Medallion architecture, Galaxy Schema) feeding curated data through a Flask API layer.
- A live, multi-source tourism dashboard combining weather, arrivals, nationalities, currency and emergency data.
- Seven curated datasets covering hotels, restaurants, museums, monuments, sites, collections and periods.
- A fully containerized, cloud-deployed application on Hugging Face Spaces.

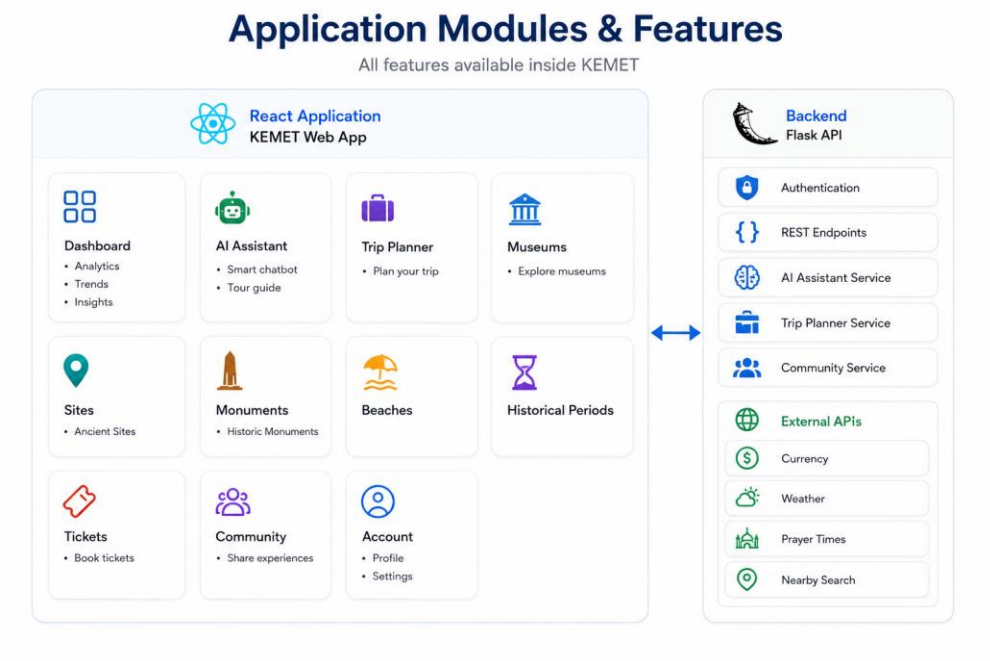


Figure 7.1 — Core features delivered by the Kemet platform

7.2 Business Impact

- Faster trip planning: travelers get grounded answers, live conditions and browsable directories in one place instead of many fragmented sources.
- Wider accessibility: native Arabic support broadens the platform's reach beyond English-only tourism tools.
- Local visibility: hotels, restaurants and cultural sites gain a structured, AI-searchable presence.
- Scalability: the Medallion/Galaxy-Schema data pipeline and Azure-backed data lake let datasets be updated and re-modeled independently of the application code.

8. Conclusion & Future Work

8.1 Conclusion

Kemet demonstrates a practical, production-style AI assistant for tourism: it pairs a general-purpose LLM (Gemini) with a lightweight, keyword-routed RAG pipeline over curated Egyptian tourism datasets, an upstream cloud-native data engineering pipeline (Azure Data Factory, Databricks, Medallion architecture and a Galaxy Schema warehouse served through Flask REST APIs), a live operational dashboard, and community/account features — all delivered through a cohesive, themed web interface and packaged for one-command Docker deployment on Hugging Face Spaces.

8.2 Future Enhancements

- Structured itinerary planning (multi-day trip builder combining sites, hotels and restaurants).
- Booking integrations with hotel/restaurant partners.
- Offline/low-connectivity mode for travelers with limited data access.
- Expanded language support beyond Arabic and English.
- Push notifications / alerts for weather or safety conditions.
- Native mobile application.
- A managed vector store to replace in-memory embedding search as datasets grow.

9. References & Acknowledgments

Repository: huggingface.co/spaces/Yousef-788/Kemet

Core technologies referenced in this documentation: Google Gemini (google-genai), sentence-transformers, Selenium, Azure Data Factory, Azure Databricks, Flask, Azure Blob Storage, Azure Cosmos DB, ddgs, Plotly, Chart.js, Open-Meteo API, Docker.

Special Thanks:

- [Mentor / Instructor Name]
- [Program Name]
- Team Members